

PROJECT REPORT
On
“Real-Time Leaderboard System”

[Python Programming Post-Lab]

Submitted By:

Muneebur Rahman [CS25D010]

Guided By:

Prof. Yogesh Katre



DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

**S. B. JAIN INSTITUTE OF TECHNOLOGY
MANAGEMENT AND RESEARCH, NAGPUR.**

(An Autonomous Institute, Affiliated to RTMNU, Nagpur)

2025-2026

© S.B.J.I.T.M.R Nagpur 2026

S.B. JAIN INSTITUTE OF TECHNOLOGY MANAGEMENT AND RESEARCH, NAGPUR

(An Autonomous Institute, Affiliated to RTMNU, Nagpur)

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

SESSION 2025-2026

CERTIFICATE

This is to certify that the Mini Project titled “**Real-Time Leaderboard System**” is a bonafide work of **Muneebur Rahman** carried out for the partial fulfillment of course work of “**Python Programming (N-SECCS401P)**”, 4th Semester, Bachelor of Technology in **Computer Science & Engineering**, S. B. Jain Institute of Technology, Management & Research, Nagpur.

Prof. Yogesh Katre
Assistant Professor
(Project Guide)

Dr. Anup Gade
Head of Department

INDEX

CERTIFICATE	i
INDEX	ii
LIST OF FIGURES	iii
CHAPTER 1 INTRODUCTION	1-2
CHAPTER 2 METHODOLOGY	3-4
CHAPTER 3 TOOLS/PLATFORMS	5-6
CHAPTER 4 DESIGN & IMPLEMENTATION	7-11
4.1 ALGORITHM	
4.2 FLOWCHART	
4.3 SOURCE CODE	
CHAPTER 5 RESULT & DISCUSSION	12-17
5.1 OUTPUT	
5.2 DISCUSSION	
5.3 APPLICATION	
CHAPTER 6 CONCLUSION	18
REFERENCES	19

LIST OF FIGURES

SR. NO.	TITLE OF FIGURE	PAGE NO.
1	Fig 4.2.1 Leaderboard flowchart	8
2	Fig 5.1.1 First page UI	12
3	Fig 5.1.2 Adding Players	12
4	Fig 5.1.3 Adding more players, ranks are displayed	13
5	Fig 5.1.4 Searching players	13
6	Fig 5.1.5 List of all players	13

CHAPTER 1

INTRODUCTION

A **Real-Time Leaderboard System** is an advanced and dynamic ranking mechanism used in modern digital platforms such as online gaming, coding competitions, educational platforms, and sports analytics. It is designed to display user rankings instantly based on their performance metrics, such as scores, points, or achievements. Unlike traditional static ranking systems, real-time leaderboards continuously update as new data is received, ensuring that users always see the most current rankings. The increasing demand for interactive and competitive environments has made leaderboard systems an essential component of many applications. For example, in online games, players expect to see their rank change immediately after achieving a new score. Similarly, in coding competitions, participants rely on real-time leaderboards to track their progress relative to others. This requires a system that is not only fast but also efficient in handling frequent updates and large volumes of data.

The main objective of this project is to design and implement an efficient real-time leaderboard system using **Heap Data Structures** and **Hashing Techniques**. Traditional sorting algorithms, such as bubble sort or merge sort, are not suitable for real-time systems because they require re-sorting the entire dataset whenever a new score is added or updated. This leads to increased time complexity and reduced performance, especially when dealing with large datasets.

To overcome these limitations, this project utilizes **heaps**, which are specialized tree-based data structures that allow efficient retrieval of the highest or lowest element. In the context of a leaderboard, a **max-heap** is typically used to quickly identify the top-ranked users. Heaps provide operations such as insertion and deletion in logarithmic time, making them ideal for systems that require frequent updates. In addition to heaps, **hashing techniques** are used to enhance performance by providing constant-time access to user data. A hash table allows the system to quickly locate and update a user's score without scanning the entire dataset. By combining heaps and hashing, the system achieves both fast updates and efficient ranking, making it highly suitable for real-time applications.

Key Features of the System

- **Real-time score updates:** The leaderboard updates instantly whenever a user's score changes.
- **Efficient ranking system:** Maintains accurate rankings without the need for full re-sorting.
- **Fast insertion and deletion:** New users can be added and existing users can be removed efficiently.
- **Retrieval of top-k players:** Quickly identifies the top performers using heap properties.
- **Scalability:** Capable of handling a large number of users and frequent updates without

performance degradation.

Background and Importance

Leaderboards play a crucial role in enhancing user engagement and motivation. They introduce a competitive element that encourages users to improve their performance. Platforms such as online games, coding contests, and fitness applications rely heavily on leaderboards to provide feedback and foster competition among users.

However, maintaining an efficient leaderboard becomes challenging as the number of users grows and updates become more frequent. A naive implementation using simple arrays or lists can lead to performance bottlenecks, especially when continuous sorting is required. This highlights the need for optimized data structures and algorithms.

By leveraging heaps and hashing, this project addresses these challenges and provides a robust solution for real-time ranking systems. The combination of these techniques ensures that the leaderboard remains fast, accurate, and scalable, even under heavy usage.

CHAPTER 2

METHODOLOGY

METHODOLOGY

The development of the Real-Time Leaderboard System is based on an efficient combination of **Max Heap** and **Hash Map (Dictionary in Python)** data structures. This hybrid approach ensures fast data access, quick updates, and efficient ranking, making it suitable for real-time applications where performance is critical.

The system is designed to handle continuous score updates while maintaining accurate rankings without the need for repeated full sorting. The methodology focuses on optimizing both time complexity and scalability.

Steps Followed

1. Storing User Data using Hash Map

All user scores are stored in a hash map, where the key represents the user ID or name and the value represents the score. This allows direct access to any user's data in constant time, eliminating the need to search through the entire dataset.

2. Maintaining a Max Heap for Rankings

A max heap is used to maintain the leaderboard rankings. In this structure, the user with the highest score is always at the root, allowing quick retrieval of the top-ranked player. The heap structure ensures that insertion and deletion operations are efficient.

3. Updating User Scores

When a user's score changes, the system performs two operations:

- The hash map is updated with the new score.
- A new entry with the updated score is inserted into the heap. Instead of modifying the existing heap entry (which is inefficient), the system adds a new entry and handles outdated ones separately.

4. Retrieving Top Players

To display the leaderboard, the system extracts the top players using heap operations. The highest scores are removed from the heap one by one to generate the top-k rankings efficiently.

5. Handling Duplicate Entries (Lazy Deletion)

Since updated scores are reinserted into the heap, older entries may still exist. To manage this, the system uses a **lazy deletion technique**, where outdated entries are ignored during extraction. The system checks the hash map to verify whether a heap entry is valid before

considering it in the ranking.

Approach

The overall approach combines the strengths of both data structures:

- **Hashing ensures $O(1)$ lookup time**, enabling quick access and updates to user scores.
- **Heap operations ensure $O(\log n)$ time complexity** for insertion and deletion, which is efficient for maintaining sorted rankings.
- The **combined approach minimizes computational overhead**, making the system capable of handling frequent updates and large datasets.

CHAPTER 3

TOOLS/PLATFORMS

1.1 SOFTWARE REQUIREMENT

The successful development and execution of the Real-Time Leaderboard System require a suitable software environment that supports efficient coding, testing, and performance optimization. This project is implemented using modern and widely used tools and technologies, ensuring reliability, ease of development, and scalability.

Programming Language: Python

The system is developed using **Python**, a high-level, interpreted programming language known for its simplicity and readability. Python is widely used in data structures and algorithm-based projects due to its clean syntax and extensive standard library support. It allows developers to implement complex logic with minimal code, making it an ideal choice for this project. Additionally, Python provides built-in modules that simplify the implementation of heaps and hash maps, which are core components of the leaderboard system.

IDE: VS Code / PyCharm

For writing and managing the code, modern Integrated Development Environments (IDEs) such as **Visual Studio Code (VS Code)** and **PyCharm** are used.

- **VS Code** is a lightweight and highly customizable code editor that supports Python through extensions. It provides features like syntax highlighting, debugging tools, and version control integration.
- **PyCharm** is a dedicated Python IDE that offers advanced features such as intelligent code completion, error detection, and built-in debugging tools, making development faster and more efficient.

Both IDEs enhance productivity and help in maintaining clean and error-free code.

Operating System: Windows

The system is designed to run on the **Windows Operating System**, which provides a user-friendly interface and wide compatibility with development tools. Windows supports Python and all required libraries efficiently, making it suitable for both development and testing purposes. However, the project can also be adapted to run on other operating systems like Linux or macOS with minimal changes.

Libraries Used

The project utilizes Python's built-in libraries to implement core functionalities:

- **heap (heapq module):**

The heapq module is used for implementing the heap data structure. Although Python provides a min-heap by default, it can be easily adapted to function as a max-heap by storing negative values. This module enables efficient insertion, deletion, and retrieval of elements, which is essential for maintaining leaderboard rankings.

- **collections (optional):**

The collections module provides specialized container data types such as dictionaries, counters, and default dictionaries. It can be optionally used to enhance data handling and simplify certain operations within the system.

DESIGN & IMPLEMENTATION

4.1 ALGORITHM

The algorithm for the Real-Time Leaderboard System is designed to efficiently manage user scores and rankings using a combination of a Hash Map and a Max Heap. This approach ensures fast updates and quick retrieval of top-ranked users without the need for full sorting after every operation.

Step-by-Step Algorithm

1. Initialization

- Create an empty hash map (dictionary) to store user names (or IDs) as keys and their scores as values.
- Initialize an empty max heap to maintain ranking order. (In Python, a min-heap is used with negative values to simulate a max heap.)

2. Processing User Operations

- For each incoming operation (such as adding or updating a score), perform the following:

a. Check if the user exists:

- If the user is new:
 - Add the user and their score to the hash map.
- If the user already exists:
 - Update the user's score in the hash map.

b. Update Heap:

- Insert a new entry into the heap as a tuple:

(-score, user)

- This ensures that the highest score has the highest priority in the heap.

3. Handling Duplicate/Outdated Entries

- Since updated scores are reinserted into the heap, older entries may still

remain.

- Apply lazy deletion:
 - While extracting elements, compare the score from the heap with the current score in the hash map.
 - If they do not match, discard the entry and continue extracting.

4. Generating Leaderboard (Top-K Players)

- To display the leaderboard:
 - Initialize an empty list for results.
 - Repeat the following until K valid players are found:
 - Extract the top element from the heap.
 - Verify its validity using the hash map.
 - If valid, add it to the result list.

5. Display Rankings

- Sort or arrange the extracted top-K players in descending order of scores.
- Assign ranks (1, 2, 3, ...) based on their position.
- Display user names along with their scores and ranks.

4.2 FLOWCHART

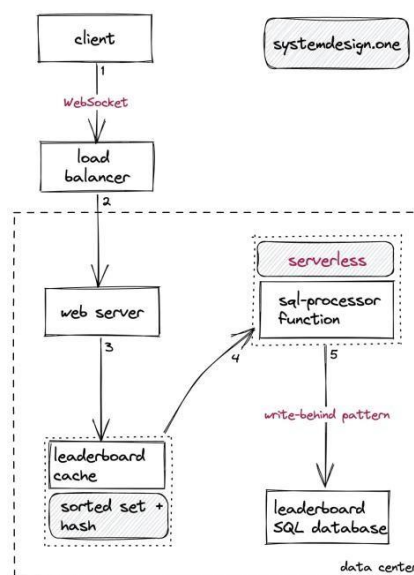


Fig 4.2.1 Leaderboard flowchart

4.3 SOURCE CODE

```
import tkinter as tk
from tkinter import ttk, messagebox

# Store data
scores = {}

def add_score():
    name = name_entry.get()
    try:
        score = int(score_entry.get())

        if name in scores:
            scores[name] += score
        else:
            scores[name] = score

        update_table()

        name_entry.delete(0, tk.END)
        score_entry.delete(0, tk.END)

    except:
        messagebox.showerror("Error", "Enter valid score")

def update_table(data=None):
    # clear table
    for row in tree.get_children():
        tree.delete(row)
```

```

# use given data or full data
display_data = data if data else sorted(scores.items(), key=lambda x: x[1], reverse=True)

# insert into table
for i, (name, score) in enumerate(display_data): tree.insert("",
    "end", values=(i+1, name, score))

def search_player():
    keyword = search_entry.get().lower()

    filtered = [(name, score) for name, score in scores.items() if keyword in name.lower()]

# sort filtered data
    filtered = sorted(filtered, key=lambda x: x[1], reverse=True)

    update_table(filtered)

def reset_table():
    search_entry.delete(0, tk.END)
    update_table()

def clear_all():
    scores.clear()
    update_table()

# Window
root = tk.Tk()
root.title("Leaderboard with Search")
root.geometry("420x400")

```

```

# Input
tk.Label(root, text="Name").pack()
name_entry = tk.Entry(root)
name_entry.pack()

tk.Label(root, text="Score").pack()
score_entry = tk.Entry(root)
score_entry.pack()

tk.Button(root, text="Add Score", command=add_score).pack(pady=5)

# Search
tk.Label(root, text="Search Player").pack()
search_entry = tk.Entry(root)
search_entry.pack()

tk.Button(root, text="Search", command=search_player).pack(pady=3)
tk.Button(root, text="Reset", command=reset_table).pack(pady=3)

tk.Button(root, text="Clear All", command=clear_all).pack(pady=5)

# Table
columns = ("Rank", "Name", "Score")
tree = ttk.Treeview(root, columns=columns, show="headings")

for col in columns:
    tree.heading(col, text=col)

tree.pack(pady=10)

root.mainloop()

```

RESULT & DISCUSSION

5.1 OUTPUT

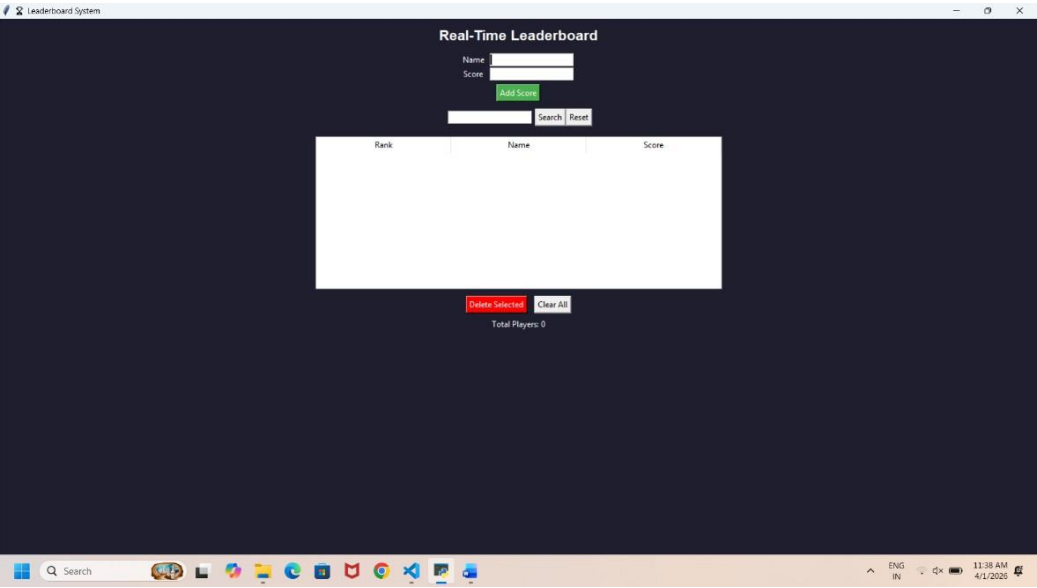


Fig 5.1.1 First page UI

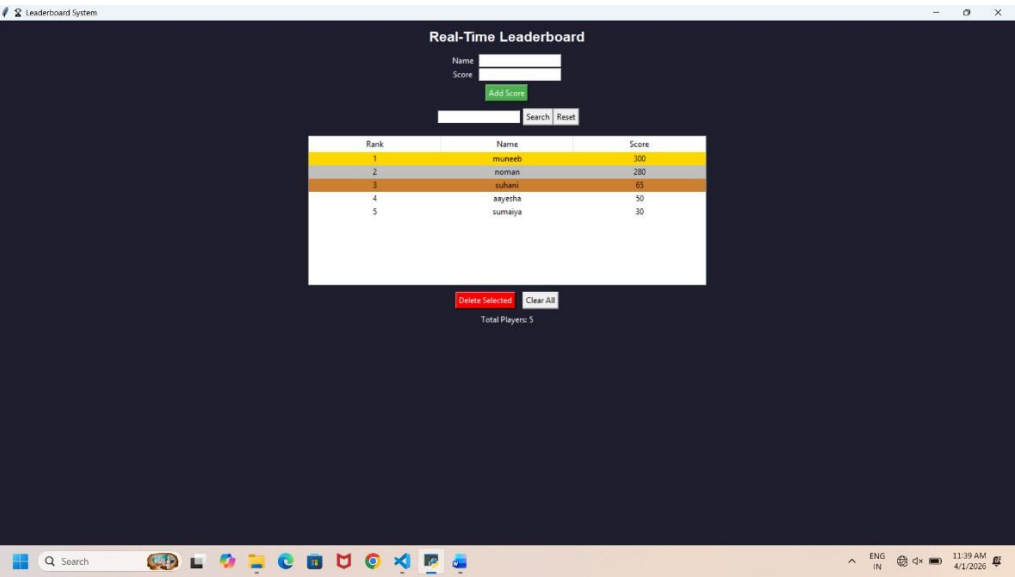


Fig 5.1.2 Adding Players

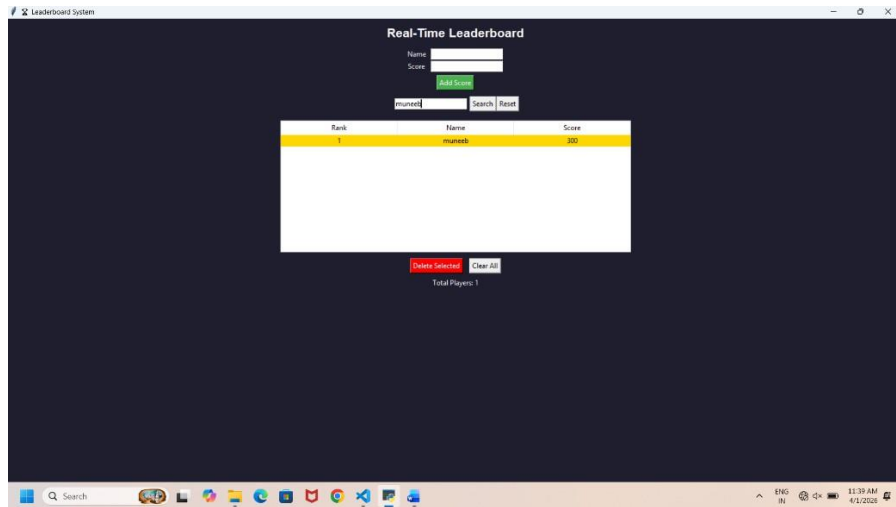


Fig 5.1.3 Adding more players, ranks are displayed

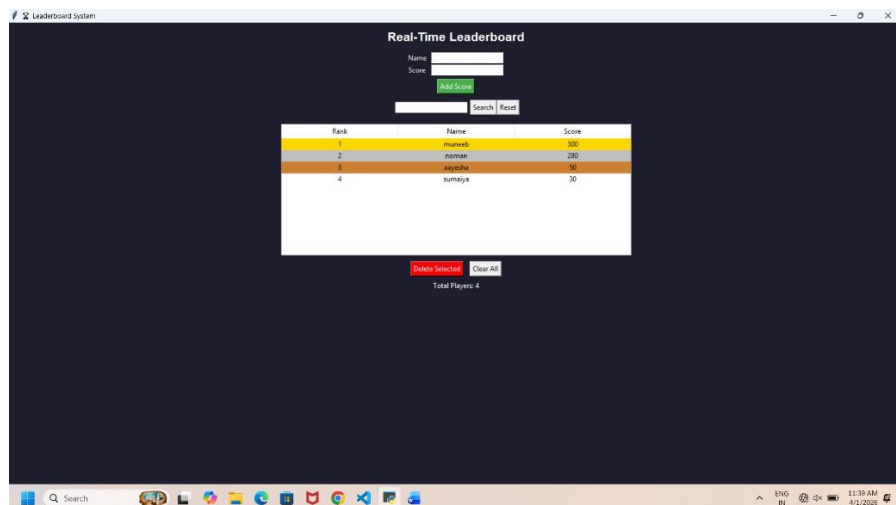


Fig 5.1.4 Searching players

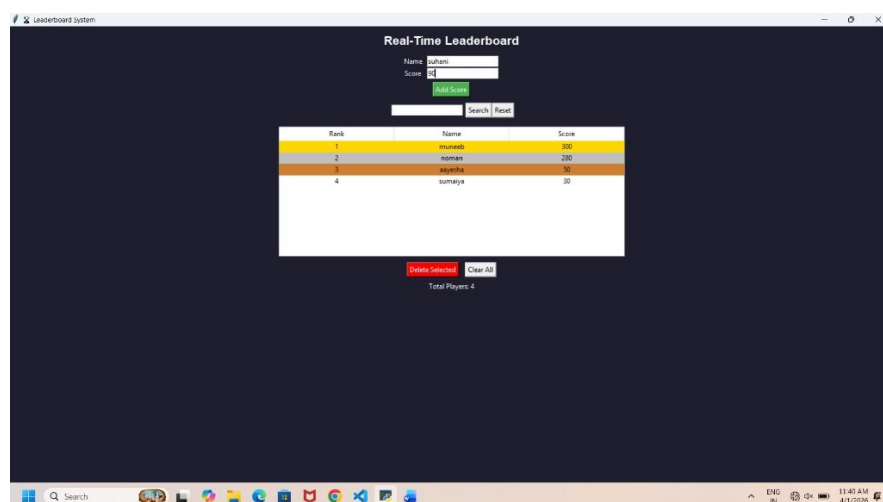


Fig 5.1.5 List of all players

5.2 DISCUSSION

The Real-Time Leaderboard System developed using Python demonstrates an efficient and scalable approach to maintaining rankings dynamically. By combining the concepts of heaps and hashing, the system overcomes the limitations of traditional sorting techniques, which are inefficient for real-time updates.

One of the major strengths of this system is its ability to handle frequent updates efficiently. In many real-world applications such as gaming and competitions, scores are updated continuously. If traditional sorting methods were used, the system would need to re-sort the entire dataset after every update, resulting in high time complexity. However, by using a heap data structure, the system ensures that insertion and retrieval operations are performed in logarithmic time, making it significantly faster.

The use of hashing (dictionary in Python) provides constant-time complexity for accessing and updating user scores. This ensures that any change in a player's score is immediately reflected in the system without delay. The combination of hashing and heap results in a highly optimized system where both update and retrieval operations are efficient.

Another important aspect of the system is the use of a **lazy deletion technique**. Since heaps do not support direct updates, duplicate entries may occur when a player's score is updated. To handle this, the system validates entries while extracting them, ensuring that only the latest and correct scores are considered. This improves accuracy without compromising performance.

During the development process, several challenges were encountered, such as handling duplicate entries in the heap, maintaining synchronization between the heap and hash map, and ensuring correct ranking order. These challenges were addressed by implementing validation checks and efficient data handling mechanisms.

Despite its advantages, the system has some limitations. It is currently implemented as a console-based application, which may not be user-friendly for non-technical users. Additionally, it does not include persistent storage, meaning the data is lost once the program is terminated. These limitations can be addressed in future enhancements.

Overall, the system performs efficiently and meets its objectives. It demonstrates the practical application of data structures in solving real-world problems and provides a strong foundation for further improvements.

5.3 APPLICATION

The Real-Time Leaderboard System has a wide range of practical applications in both everyday life and professional environments, especially in areas where ranking, competition, and performance tracking are essential.

One of the primary applications of this system is in **online gaming platforms**. In modern multiplayer games, players compete with each other, and their scores or achievements are continuously updated. A real-time leaderboard helps display the top players instantly, motivating users to improve their performance. It enhances user engagement and creates a competitive environment by showing rankings dynamically without delay.

Another important application is in **coding competition platforms** such as hackathons and online judges. Websites that host programming contests require efficient ranking systems to display participants' scores based on problem-solving performance. Since submissions and scores are updated frequently, a real-time leaderboard ensures accurate and fast ranking updates. This helps participants track their position instantly and encourages healthy competition among programmers.

The system is also widely used in **sports ranking systems**. In tournaments, leagues, or events, teams and players are ranked based on their performance, points, or scores. A real-time leaderboard allows organizers and viewers to monitor rankings as the game progresses. This is particularly useful in live events where rankings change frequently, providing an engaging experience for audiences.

In the field of **e-learning platforms**, leaderboard systems are used to track student performance. Many educational platforms incorporate quizzes, assignments, and gamified learning modules. A leaderboard helps rank students based on their scores, encouraging active participation and improving learning outcomes. It also helps instructors analyze student performance and identify top performers.

Another growing application is in **social media and engagement platforms**. Leaderboards can be used to rank users based on activity levels such as likes, shares, comments, or content contributions. This encourages users to be more active and engaged on the platform. Influencers, content creators, and active users can be recognized through such ranking systems.

Additionally, the system can be applied in **corporate environments** for employee performance tracking, sales leaderboards, and productivity monitoring. Companies can use leaderboard systems to motivate employees by ranking them based on performance metrics, thereby increasing efficiency and competitiveness within teams.

Overall, the Real-Time Leaderboard System provides a fast, efficient, and scalable solution for ranking and performance tracking across various domains. Its ability to handle continuous updates and instantly reflect changes makes it highly valuable in real-world applications.

CONCLUSION

The Real-Time Leaderboard System successfully demonstrates the use of advanced data structures such as heaps and hashing to efficiently manage dynamic rankings. The project achieves its primary objective of maintaining a real-time leaderboard where scores are continuously updated and rankings are displayed instantly.

The implementation highlights the importance of selecting appropriate data structures for solving specific problems. By using a hash map, the system ensures fast access and updates of user data, while the heap structure enables efficient retrieval of top-ranked players. This combination results in a highly optimized system with improved performance compared to traditional approaches.

The project also provides valuable insights into algorithm design and optimization. It shows how time complexity can be reduced by avoiding unnecessary operations such as repeated sorting. The use of techniques like lazy deletion further enhances the efficiency and correctness of the system.

From a learning perspective, this project helps in understanding key programming concepts such as data structures, algorithm design, and problem-solving strategies. It also demonstrates how theoretical concepts can be applied to build practical applications.

Although the current implementation is effective, there is scope for further improvement. The system can be enhanced by adding a graphical user interface (GUI) to improve usability. Additionally, integrating a database can enable permanent storage of data, allowing the system to retain information even after restarting. Features such as user authentication, real-time synchronization using APIs, and cloud-based storage can also be added to make the system more robust and scalable.

In conclusion, the Real-Time Leaderboard System is a practical and efficient solution for managing rankings in dynamic environments. It serves as a strong foundation for developing more advanced systems and demonstrates the power of data structures in real-world applications.

REFERENCE

□ Introduction to Algorithms –

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein (Used for understanding Heap and data structures concepts)

□ Data Structures and Algorithms in Java –

Michael T. Goodrich, Roberto Tamassia

(Reference for Java-based data structures implementation)

□ Java Collections Framework –

Oracle Documentation

<https://docs.oracle.com/javase/8/docs/technotes/guides/collections/>

(Used for PriorityQueue and HashMap)

□ Design A Leaderboard –

<https://leetcode.com/problems/design-a-leaderboard/>

(Problem inspiration and logic reference)

□ Heap Data Structure –

<https://www.geeksforgeeks.org/heap-data-structure/>

□ Hash Table –

<https://www.geeksforgeeks.org/hashing-data-structure/>

□ GeeksforGeeks –

<https://www.geeksforgeeks.org/>

(Used for understanding concepts and examples)

□ W3Schools –

<https://www.w3schools.com/>

(Basic Java reference)